ERLANG / OTP

COTA HERNANDEZ CHRISTIAN ARMANDO - 23211941
CRUZ SANCHEZ KEVIN ALFREDO - 23211942
CUEVAS MARQUEZ PABLO ANGEL - 23211943
DEL ANGEL DEL ANGEL EMMANUEL - 21210366

CONTEXTO E HISTORIA

Año y Origen: Creado en 1986 dentro del Laboratorio de Ciencias de la Computación de Ericsson.
Liberación: Publicado como código abierto (Open Source) en 1998.
Creadores: Joe Armstrong, Robert Virding y Mike Williams.
El problema a resolver: Desarrollar software para conmutadores telefónicos de la red de Ericsson. Requerían manejar miles de procesos concurrentes sin detener jamás el sistema (cero tiempo de inactividad).



LET IT CRASH

Filosofía principal

Let it crash" (Deja que falle). Se asume que los errores en producción son inevitables

Aislamiento

Los procesos en Erlang son completamente independientes y no comparten memoria.

01 Recuperación

Si un proceso falla, muere de inmediato sin corromper al resto del sistema.

02 Supervisores

Un programa supervisor detecta la caída y reinicia el proceso afectado desde un estado limpio y estable.

03 Resultado

Permitió alcanzar una disponibilidad histórica de "nueve nueves" (99.9999999% de tiempo en línea) en telecomunicaciones.

MODELO DE COMPUTO

01

Asignación única

- Una variable se liga una sola vez
- El = es pattern matching, no asignación

erlang
```
X = 5.   % ok
X = 6.   % error: no match
```

02

Recursión de cola (no hay ciclos)

- Sin for ni while: la función se llama a sí misma
- Con un acumulador, la pila no crece

erlang
```
sum(L) -> sum(L, 0).
sum([], Acc)    -> Acc;
sum([H|T], Acc) -> sum(T, Acc + H).
```

03

Paso de mensajes + shared nothing

- Cada proceso tiene su propia memoria
- Se comunican solo con mensajes (se copian)
- Sin locks; si uno falla, los demás siguen

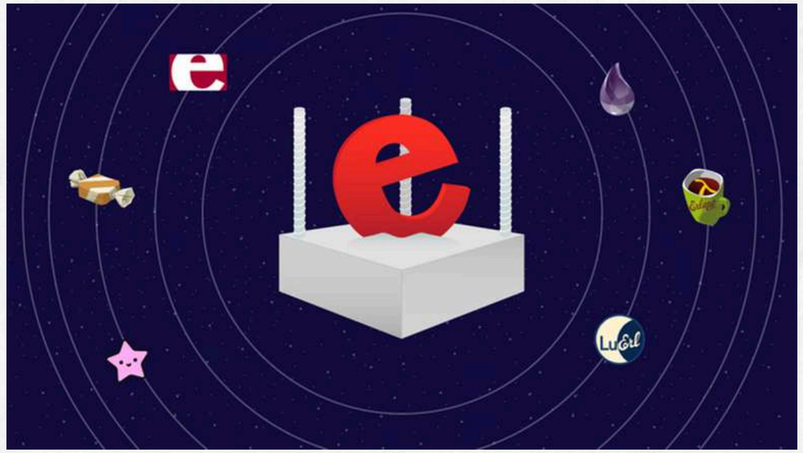
erlang
```
Pid ! hola.            % enviar
receive M -> M end.    % recibir
```

04

Evaluación estricta

- Los argumentos se calculan antes de llamar a la función

SINTAXIS BASICA



1. Datos básicos

- Variable: Mayúscula. Átomo: minúscula (valor fijo)
- Tupla { } y lista [ ]

erlang
```
Edad  = 20.          % variable
Estado = ok.         % átomo
Punto = {3, 4}.      % tupla
[H|T] = [1,2,3].     % H = 1, T = [2,3]
```

2. Funciones con cláusulas

- Cláusulas separadas con ;, la última termina con .
- when agrega una condición (guard)

erlang
```
mayor(A, B) when A > B -> A;
mayor(_, B)            -> B.
```

3. Funciones anónimas

- fun ... end

erlang
```
lists:foreach(fun(N) -> io:format("~w~n",[N]) end, lists:seq(1,10)).
```

SISTEMA DE TIPOS

```
% 1. Tipado Fuerte: Error inmediato en Runtime (Bad Arithmetic)
1 + "hola".
% -> ** exception error: bad argument in an arithmetic expression

% 2. Dialyzer: Definición explícita de tipos (-spec)
-spec sumar(integer(), integer()) -> integer().
sumar(A, B) -> A + B.
% Si llamamos sumar(5, "hola"), Dialyzer detecta el error sin ejecutar el programa.
```

- Tipado Dinámico y Fuerte: El tipo de dato se verifica en tiempo de ejecución. No hay conversión implícita no deseada (un entero no se suma automáticamente con un string).
- Modelo de Evaluación: Estricto (Eager evaluation) — los argumentos de una función se evalúan completamente antes de ejecutar la función.
- Dialyzer (Análisis Estático Opcional): Herramienta que utiliza Discrepancy Analyzer for Erlang. Permite encontrar errores de tipo antes de ejecutar el código mediante supertipado de éxito (success typings), manteniendo la flexibilidad del lenguaje dinámico.

ARQUITECTURA DEL RUNTIME (LA BEAM)

La Máquina Virtual BEAM: Bogdan's Erlang Abstract Machine. Diseñada desde cero para concurrencia masiva, baja latencia y alta disponibilidad.

Procesos Ultraligeros (No hilos del Sistema Operativo):

- Los procesos son gestionados directamente por la BEAM.
- Consumen apenas ~300 bytes de memoria inicial por proceso.
- Una sola máquina puede ejecutar millones de procesos simultáneos sin colapsar.

Aislamiento Total (Shared Nothing): Cada proceso tiene su propio heap y su propio recolector de basura (Garbage Collector). Si un proceso falla, no interrumpe a los demás ni invalida la memoria general.

```
% Crear (spawn) 1,000,000 de procesos concurrentes en segundos:
lists:foreach(
  fun(Id) ->
    spawn(fun() -> io:format("Proceso ~p activo~n", [Id]) end)
  end,
  lists:seq(1, 1000000)
).

% Resultado: ~300 MB de RAM en total.
% (En Java/C++ con hilos de SO requeriría cientos de Gigabytes).
```

CASO DE PRODUCCIÓN: WHATSAPP

- Manejo exitoso de ~2 millones de conexiones simultáneas por servidor.
- Infraestructura mantenida por un equipo de solo 50 ingenieros, aprovechando los procesos ligeros y la alta concurrencia de Erlang.
- La Familia BEAM: Lenguajes modernos como Elixir y Gleam operan hoy en día sobre esta misma máquina virtual.



REFERENCIA

- [1] E. Letuchy, "Erlang at Facebook and WhatsApp," IEEE Software, vol. 31, no. 1, pp. 71-75, 2014

MUCHAS GRACIAS